Cortex Cloud Runtime Security Documentation

1 | Cortex CLI

Abstract

The Cortex CLI is a unified command-line tool integrating Cloud Workload Protection, API Security, and Code Security scans into a single executable.

The Cortex CLI is a unified command-line tool that integrates scanning for Cloud Workload Protection (CWP), API Security (WAAS), and Code Security (AppSec). From a single binary, security teams can enforce organizational policies and proactively detect vulnerabilities, misconfigurations, and exposed secrets across source code, container images, and API specifications.

Scope: The Cortex CLI evaluates findings against Unified Application Security Policies and returns structured results with policy correlation, severity breakdowns, and remediation guidance. The Cortex CLI does not create, edit, or delete policies; all policy management operations are performed through the Cortex Cloud tenant or the public API.

Primary use cases

The CLI supports the following primary workflows:

- Local code development (AppSec): Enable developers to detect hardcoded secrets, IaC misconfigurations, and vulnerable dependencies directly from their terminal before committing code
- CI/CD automation: Embed security checks into build scripts (such as Jenkins, GitHub Actions) to automatically detect issues and enforce security gates during the build process
- Container Workloads (CWP): Integrate container scanning directly into CI builds to detect vulnerabilities and malware before images are pushed to production registries
- API Testing: Evaluate application endpoints for high-risk vulnerabilities and specification leaks as a standard step prior to deployment

Core capabilities

The Cortex CLI consolidates multi-domain security scanning into a single executable tool:

- Unified scanning engine: Integrates native scanning for Cloud Workload Protection (CWP), API Security, and Code Security. A single set of global flags controls authentication, output format, upload behavior, and error handling across all scan types
- Code security: Detects hardcoded secrets, Infrastructure-as-Code (IaC) misconfigurations, and open-source dependency vulnerabilities (SCA) directly within developer environments. The SCA scanner generates Software Bills of Materials (SBOMs) for supply chain compliance
- Container security (CWP): Generates Software Bill of Materials (SBOMs) and detects vulnerabilities or malware in container images before registry push. Container scanning integrates directly into CI builds to prevent vulnerable images from reaching production registries
- API risk validation: Identifies vulnerabilities, sensitive data leaks, and configuration errors by analyzing OpenAPI and Swagger specifications. API testing validates application endpoints for high-risk vulnerabilities and specification leaks as a standard step prior to deployment
- Automated security guardrails: Enforces compliance directly within CI/CD pipelines by dynamically blocking deployments that violate organizational security policies

Prerequisites

Before installing and running the Cortex CLI, verify that your environment and account meet the following system and access requirements:

Prerequisite	Description
License	An active Cortex Cloud license with the Application Security add-on for Code Security if required
Permissions	The API key must be associated with a user or role that has CLI Tools permissions: • View: grants read-only access (sufficient for `--upload-mode no-upload`). **NOTE:** This role is not supported for CWP, as the CWP system does not support offline mode. • View/Edit: grants full access including scan result upload (required for `--upload-mode upload` and `--upload-mode no-code`) **NOTE:** There are no preconfigured CLI-specific roles. Add the CLI Tools permission to an existing role or create a dedicated custom role.

1.1 | Connect Cortex CLI

Connect Cortex CLI to scan supported Cortex Cloud modules and gain insights into your security posture, enabling you to identify, analyze and address potential risks.

You can choose from three main installation workflows:

- Package Manager: The most efficient developer workflow, utilizing Homebrew for macOS/Linux and Scoop for Windows
- Manual download: Directly download the binaries for any operating system
- UI-based installation: Onboard and download the CLI directly from your tenant

> **PREREQUISITE:**

- **System requirements**:
  - **macOS** (Intel Core i7, such as Sequoia): To ensure all functionalities work correctly, you must install the `vectorscan` dependency via **Homebrew**, using this command: `brew install vectorscan`
  - **RHEL 8.10** and **Red Hat UBI9**. The following prerequisites must be met:
    - Install `patchelf`
    - Install `zstd`
  - **Ubuntu 20** requires the `prefetch` utility
  - **Ubuntu (for linux-amd64)** also requires the `libhyperscan5` library. To install, run `sudo apt install libhyperscan5`
  - **Linux for AppSec Module**: Support is provided for systems meeting the following specifications:
    - **RHEL 10**: Kernel: 6.12, glibc: 2.39
    - **Debian 12**: Kernel: 6.1.27, glibc: 2.36
    - **Ubuntu**:
      - Version 18.04 - Kernel: 4.15, glibc: 2.27
      - Version 20.04 - Kernel: 5.4, glibc: 2.31
      - Version 22.04 - Kernel: 5.15, Glibc: 2.35
      - Version 24.04 - Kernel: 6.8, Glibc: 2.39
  - Windows: AMD 64 and ARM 64
- **For cURL-based downloads**:
  - `curl`
  - `jq`
    - On **Ubuntu/Debian-based Linux** distributions: `sudo apt-get install jq`
    - On **RedHat/CentOS/Fedora**: `sudo yum install jq`
    - **macOS** (using `Homebrew`): `brew install jq`
    - **Windows**:
      - Download the executable from jq GitHub releases
      - If `Chocolatey` is installed: `choco install jq`
- **Permissions**:
  - **With upload results**: Requires a role with `CLI View/Edit` (write) permissions.
  - **Local scan only**: Requires a role with `CLI Read Only` (read-only) permissions

  For more information refer to Cortex CLI.
- **Roles**: There are no out-of-the-box CLI roles. The CLI authenticates via an API key. Ensure the API key associated with your role includes the required permissions

- **API Security level**: The API key must be set to the `Standard` security level. CLI scans will fail if the security level is set to `Advanced`
- **Best practice** (required for SCA vulnerability suppression):
  - Run the CLI within your current working directory (<current_directory_path>). It is recommended to use the absolute file path for your current working directory
  - Ensure that the `--repo-id` parameter includes the `<repo_owner_name>/<repo_name>` structure, with the `<repo_name>` matching the exact name of the directory

    Example 161. Example

    The present working directory is `Users/test/<repo_name>`. Therefore, the `--repo-id` parameter must be `--repo-id <repo_owner_name>/<repo_name>`, ensuring that `<repo_name>` precisely matches the directory name within the structure.

  - For terminal actions performed by Cortex Cloud IDE extensions on Windows, Command Prompt (CMD) is the supported environment. PowerShell is not supported for these actions

Workflow 1: Install through a Package Manager

Using a package manager is the recommended method for installing the Cortex CLI. Use `Homebrew` (for macOS and Linux) or `Scoop` (for Windows).

macOS & Linux (Homebrew)

Supported on macOS (Apple Silicon & Intel) and Linux (x86_64 & arm64).

Requires Homebrew.

- Standard installation

```
brew tap paloaltonetworks/cortexcli
brew install cortexcli
cortexcli --version
```

- Pinning to a specific version (optional)

  If your workflow requires a specific version, use one of the following methods instead:

  - Pin to a release line (for example stay with v 0.18.x)

    Use this method to lock the CLI to a specific minor version but still receive automatic security patches.

```
brew install cortexcli@0.18
# keg-only — add to PATH if needed:
echo 'export PATH="$(brew --prefix cortexcli@0.18)/bin:$PATH"' >> ~/.zprofile
```

  - Pin to an exact version (for example exactly 0.18.0):

    Use this method to strictly lock the CLI to a precise build. This prevents all automatic updates.

Windows (Scoop)

Supported on Windows x64.

Requires Scoop.

- Standard installation

  ```
  scoop bucket add cortexcli https://github.com/PaloAltoNetworks/homebrew-cortexcli
  scoop install cortexcli
  cortexcli --version
  ```

- Install a specific version (optional)

  If your workflow requires a specific version, use this method instead:

  ```
  scoop install cortexcli@0.18.0
  ```

Workflow 2: Manual download (any OS)

You can manually download the binaries for macOS, Linux, or Windows.

1. Step 1: Download the binary.

   Retrieve the specific archive for your platform from the releases page.

   - macOS / Linux: Download the appropriate `.tar.gz` archive for your system architecture
   - Windows: Download the appropriate `.zip` archive

2. Step 2: Verify and Extract.

   Verify the download against the `SHA256SUMS` file provided on the releases page, then extract the archive.

   - macOS / Linux: The extracted executable will be named `cortexcli`
   - Windows: The extracted executable will be named `cortexcli.exe`

3. Step 3: Add to PATH.

   Place the extracted file in a directory that is included in your system's `PATH` so you can run it from any terminal.

   - macOS / Linux: Move cortexcli to a directory such as `/usr/local/bin/`
   - Windows: Move `cortexcli.exe` to a dedicated folder and add that folder's path to your system's Environment Variables

Workflow 3: UI-Based Installation

This method allows you to install the CLI directly from your tenant. Instead of downloading a standard installation file through your web browser, the Cortex UI generates a custom installation command that you must run in your terminal to securely pull and authenticate the CLI binary.

Step 1: Generate the installation command (in the UI)

1. On your tenant.

   a. Navigate to Settings → Data Sources → + Data Source.

   b. Enter Cortex CLI in the search bar → Hover over the Cortex CLI card → Connect or Connect Another Instance.

2. On the Configure step of the integration wizard, select your operating system from the menu and click Next.

3. On The Authenticate step of the wizard.

    a. Generate an API:

        i. Select Generate API key. Permission options:

            - With upload results permissions. Creates a CLI role for the API key with CLI View/Edit options. It is recommended as it grants the API key permissions to not only access data, but also to upload or send data back
            - If you do not select this option, the generated API key creates a CLI Read Only role with CLI View permissions only

        > **NOTE:**
        >
        > The Cortex CLI requires an API key with the `Standard` security level.

        ii. Copy and save the the generated `API Key ID` and `API key` that are displayed in their respective fields.

        iii. Copy and save the the generated API key from the Retrieve your API key field.

        The UI generates and displays a code command. Copy this provided code block.

        > **NOTE:**
        >
        > On macOS arm 64 architecture you must unpack the downloaded file to retrieve the executable.

        iv. Verify that the generated API key is displayed under the API Keys inventory.

Step 2: Download the CLI (in your terminal).

Before running the command, you may need to insert your specific credentials into the code you just copied:

1. If the code contains placeholders, replace `${API_KEY}` with the API key you saved.

2. If needed, retrieve your public API URL by navigating to Settings → Configurations → API Keys and clicking Copy API URL, then paste it into the code.

3. Paste the finalized copied command into your local terminal and press Enter. The command you are running uses the following underlying syntax:

    ```
    curl -k -u $CORTEX_API_ID::$CORTEX_API_KEY --output ./cortexcli $CORTEX_FQDN/api/v2/remote-li/{version}/{platform}/artifacts
    ```

    What this does: This securely connects to your specific Cortex tenant (`$CORTEX_FQDN`) using your newly generated API credentials and downloads the `cortexcli` application directly to your current folder.

Step 3: Make the CLI Executable (macOS & Linux only).

By default, macOS and Linux restrict downloaded files from running as programs. You must explicitly grant the downloaded file permission to execute by running:

```
chmod +x cortexcli
```

Step 4: Verify the installation.

Confirm that the CLI was downloaded and authenticated successfully by asking it to report its version. The command you use depends on where the file is currently located:

- If you moved the file to a directory on your system PATH:

  ```
  cortexcli -v
  ```

- If the file is still in your current download folder (not in your system PATH):

  ```
  ./cortexcli -v
  ```

If the terminal displays the version number, the installation is complete and the CLI is ready to use. You can now return to the Cortex Cloud UI and click Done.

Post-installation actions

Use the following commands to manage your CLI application lifecycle after the initial installation.

macOS and Linux

- Upgrade to latest version

  ```
  brew upgrade cortexcli
  ```

- Freeze whatever you have now (blocks `brew upgrade` from touching it)

  ```
  brew pin cortexcli
  ```

- Uninstall

  ```
  brew uninstall cortexcli
  ```

Windows

- Upgrade to latest version

  ```
  scoop update cortexcli
  ```

- Prevent upgrades

  ```
  scoop hold cortexcli
  ```

- Allow upgrades again

  ```
  scoop unhold cortexcli
  ```

- Uninstall

  ```
  scoop uninstall cortexcli
  ```

Troubleshooting

`cortexcli --version` shows a different version than I just installed

You likely have an older copy of `cortexcli` earlier on your PATH — for example from the macOS `.pkg` installer, a manual download, or a previous tenant download. The shell is finding that one first.

Find every copy:

`macOS / Linux which -a cortexcli`

`Windows (PowerShell) where.exe cortexcli`

Expected location for the package-manager install:

- macOS (Homebrew): `/opt/homebrew/bin/cortexcli` or `/usr/local/bin/cortexcli`
- Linux (Homebrew): `/home/linuxbrew/.linuxbrew/bin/cortexcli`
- Windows (Scoop): `%USERPROFILE%\scoop\shims\cortexcli.exe`

Remove the old copy:

- macOS `.pkg` installer → `sudo rm /usr/local/bin/cortexcli`
- Manual / tenant download → delete the binary at the path shown by `which -a` / `where.exe`
- Windows installer → uninstall via Settings → Apps → Installed apps, or delete the `.exe` shown by `where.exe`

Finalize: Open a new terminal (so the shell drops its command cache) and re-run `cortexcli --version`.

1.2 | Authenticate credentials

After installation, you must configure your credentials. The installer does not save them for you. You can authenticate the Cortex CLI using one of these methods: command-line flags, configuration file or environment variables.

Method 1: Configuration file

(Recommended for local use). For persistent local authentication without typing credentials every time, use a configuration file. The CLI natively reads this file when executing commands, so you do not need to manually source or load it into your shell environment.

File location: The CLI looks for a file named `cortex.env` in your home directory or the current working directory.

Format: `KEY=VALUE` pairs.

- MacOS / Linux setup

  Create an environment configuration file: Instead of using flags, create an environment configuration file named `cortex.env`. Save this file in your home or working directory and add your credentials as variables.

  ```
  cat > ~/cortex.env << EOF
  CORTEX_API_BASE_URL=https://api-<TENANT>.xdr.us.paloaltonetworks.com
  CORTEX_API_KEY=<YOUR_API_KEY>
  CORTEX_API_KEY_ID=<YOUR_KEY_ID>
  EOF
  chmod 600 ~/cortex.env
  ```

- Windows setup

  ```
  $configPath = "$env:USERPROFILE\cortex.env"
  @"
  CORTEX_API_BASE_URL=https://api-<TENANT>.xdr.us.paloaltonetworks.com
  CORTEX_API_KEY=<YOUR_API_KEY>
  CORTEX_API_KEY_ID=<YOUR_KEY_ID>
  "@ | Out-File -FilePath $configPath -Encoding UTF8
  ```

Method 2: Environment variables

(Recommended for CI/CD): Best practice for automation (Jenkins, GitHub Actions, GitLab) to avoid committing secrets to code.

Variable Name	Description
`CORTEX_API_BASE_URL`	Your tenant URL (such as https://api-example.xdr.us.paloaltonetworks.com)
`CORTEX_API_KEY`	The secret key token
`CORTEX_API_KEY_ID`	The ID associated with the key

Example 162. Example exports

Linux / macOS (Bash/Zsh):

```
export CORTEX_API_BASE_URL="https://api-tenant.xdr.us.paloaltonetworks.com"
export CORTEX_API_KEY="secret-key-123"
export CORTEX_API_KEY_ID="1"
```

Windows (PowerShell):

```
$env:CORTEX_API_BASE_URL="https://api-tenant.xdr.us.paloaltonetworks.com"
$env:CORTEX_API_KEY="secret-key-123"
$env:CORTEX_API_KEY_ID="1"
```

Method 3: Command-line flags (overriding)

Use for one-off scans or testing different keys. These flags must appear in the `[global flags]` position (before the **module** name).

Syntax:

```
cortexcli --api-base-url <URL> --api-key <KEY> --api-key-id <ID> code scan ...
```

1.3 | Cortex CLI usage

To execute a Cortex CLI scan, run:

```
cortexcli [global flags] [module name] scan [module flags]
```

Command breakdown

- `cortexcli`: The Cortex CLI binary - a global option. Establishes the execution environment for all subsequent commands
- Global flags: Flags that apply across all supported modules. Place global flags between `cortexcli` and the `module` name
    - `--api-base-url <value>`
    - `--api-key <value>`
    - `--api-key-id <value>`

  Additional global flags are supported by the AppSec and CWP modules, but not the WAAS module. Refer to Cortex CLI common command line reference guide for more information.
- Module name: Select the module (environment) to be scanned:
    - `api` — API Security. For more information about API Security scans, refer to Cortex CLI for API Security
    - `image` — CWP. For more information about CWP scans, refer to Cortex CLI for Cloud Workload Protection
    - `code` — Cortex Cloud Application Security. For more information about Cortex Cloud Application Security refer to Cortex CLI for Code Security
- Module flags: The flags available for the selected command:
    - For flags common to all environments, refer to Cortex CLI common command line reference guide
    - For flags specific to CWP refer to Cloud Workload Protection command line reference
    - For flags specific to API Security, refer to Cortex CLI API Security command line reference guide
    - For flags specific to Cortex Cloud Application Security, refer to Cortex CLI Cortex Cloud Application Security command line reference

Examples

Global flags: Apply to all modules. Place between cortexcli and the module name:

```
# Authenticate and scan with global authentication flags
cortexcli --api-base-url https://api.xdr.us.paloaltonetworks.com --api-key <KEY> --api-key-id <KEY_ID> code scan
--directory .
```

Global flags common to AppSec and CWP: Upload mode, exit code handling, and log output. Not supported by WAAS:

```
# Run an AppSec scan in no-upload mode with soft-fail and log output
cortexcli --upload-mode no-upload --soft-fail --no-fail-on-crash --log code scan --directory .
```

AppSec scan: Scan source code for IaC misconfigurations, SCA vulnerabilities, and secrets:

```
# Scan a repository directory and filter results to critical and high severity
cortexcli --upload-mode no-upload code scan --directory /path/to/repo --severity critical,high
```

CWP scan: Scan a container image for vulnerabilities:

```
# Scan a container image with soft-fail enabled
cortexcli --soft-fail image scan --image myapp:latest
```

API Security scan: Scan APIs for security issues. Global flags (other than authentication) are not supported:

```
# Run an API Security scan
cortexcli api scan --api-spec /path/to/openapi.yaml
```

> **NOTE:**
>
> - For more information about CLI usage for CWP, refer to Cortex CLI for Cloud Workload Protection
> - For more information about CLI usage for API Security, refer to Cortex CLI for API Security
> - For more information about CLI usage for Cortex Cloud Application Security, refer to Cortex CLI usage for Cortex Cloud Application Security

1.4 | Self-service API keys for CLI scans

This self-service model allows developers to programmatically generate task-specific keys for CLI and IDE scans via the Public API. By using a Primary API key as a master credential, developers can provision restricted-access keys, such as `read-only` for local scans, without requiring administrative permissions in the UI. This approach maintains tenant security by ensuring all scans follow the principle of least privilege.

Prerequisite: You must have sufficient administrative permissions within your tenant to create new roles and manage API keys.

IMPORTANT: When generating an API key, ensure you select the Standard security level. CLI scans will fail if the security level of the API key is set to Advanced.

Create custom roles

Navigate to your role management settings in the tenant to generate the following three roles with these exact permission sets.

Role Name	Required Permission And Description
CLI Read-Only Custom	CLI Tools View: Grants permission to run CLI scans and view output locally without uploading results to the tenant
CLI Write Custom	CLI Tools View/Edit: Grants permission to run CLI scans and upload/manage results within the tenant
Public API (PAPI) Edit	Public API View/Edit: Grants the administrative permission required to programmatically generate and manage new API keys

Assign roles to a privileged user

To establish a Primary key holder, you must grant a specific privileged user the permissions from all three custom roles. Because the UI allows only one role to be assigned directly to a user, you must use User Groups to grant multiple roles simultaneously.

1. Create user groups: Create three separate User Groups in your tenant, assigning one of the custom roles to each group.
2. Add user to groups: Add the designated privileged user to all three of these User Groups.
3. Verify accumulated permissions: Edit the primary user and ensure that the User Groups field includes the three user groups.

This user now has the combined authority to generate the Primary API Key required to set up programmatic key generation.

Generate and use API keys

The designated privileged user must manually generate a Primary API Key through the console. This key must be associated with the `CLI Read-Only Custom`, `CLI Write Custom`, and `Public API (PAPI) Edit` roles. The primary key acts as the master credential for subsequent automation.

Using the Primary Key, developers can now make calls to the Public API to generate subsequent keys as needed for IDE or CLI scans:

- To run scans without uploading the results to the platform: Generate a key and associate it only with the `CLI Read-Only Custom` role.
- To run scans and upload the results to the platform: Generate a key and associate it only with the `CLI Write Custom` role.

The following `curl` command demonstrates how developers can use the Primary Key to generate a new API key assigned with the `CLI Read-Only Custom` role:

```
 curl --request POST \
  --url https://api-viso-k2ibu8behynsxbzuncdau6.xdr-qa2-
uat.us.paloaltonetworks.com/public_api/v1/api_keys/generate \
  --header 'Accept: application/json' \
  --header 'Content-Type: application/json' \
  --header 'authorization: <YOUR_PRIMARY_KEY_HERE>' \
  --header 'x-xdr-auth-id: <YOUR_AUTH_ID>' \
  --data '{
  "request_data": {
    "roles": [
      "CLI Read-Only Custom"
    ],
    "security_level": "standard",
    "comment": "Developer CLI Read-Only scan key",
    "expiration": 1773147108
  }
}'
```

For more information about generating API Keys, refer to Manage API keys.

To ensure the keys are configured correctly, privileged users can verify their status by navigating to Settings → API Keys. Locate the generated key in the API Keys inventory and confirm that the Role column reflects the specific custom role assigned rather than a broad administrative role.

1.5 | Cortex CLI common command line reference guide

This reference guide describes the command line flags used to manage the Cortex Cloud Application Security, Cloud Workload Protection (CWP), and API Security modules through the Cortex CLI. It includes **common flags**, which apply to all supported modules, and **global flags**, which are shared specifically across AppSec and CWP and must be placed before the command.

In instances where the same flag is available in both categories, its underlying functionality remains identical; however, its required placement within the command structure differs depending on how it is used.

Common flags

The following table describes CLI commands common to all supported Cortex CLI modules. These flags are typically used **after** the module and command.

Command	Description
--api-base-url $CORTEX_API_BASE_URL	The public facing API URL. To retrieve the URL, under Settings, select Configurations → API Keys → copy API URL. Required: true.
--api-key $CORTEX_API_KEY	The API key used for authorization. Required: true.

Command	Description
--api-key-id $CORTEX_API_KEY_ID	The API key ID. Required: true.
--soft-fail $CORTEX_SOFT_FAIL	Identifies and reports errors identified during a scan but does not trigger a failing condition, allowing CI/CD pipelines to continue without disruption. Instead of failing the build, the scan returns a successful exit code of `0`. Unlike skipped or suppressed checks, soft fail errors are still reported but do not cause the scan to fail. Required: false. Unlike skipped or suppressed checks, soft fail errors are still fully reported. For soft fails, a failed check matches the defined severity threshold. If multiple soft fail severities are specified, the highest severity acts as the threshold. Fundamental execution errors (such as exit codes 126 or 127) are not suppressed and will still fail the build.
--support	Enable debug logs and upload the logs to the platform. Usage: Before the module name. For example: `cortexcli --api-base-url <URL> --api-key <KEY> --api-key-id <ID> --support code scan --directory . --upload-mode no-upload --repo-id my/test --branch test`
--log-level	Set the logging level (INFO, WARNING, ERROR) for Stdout output
--http-proxy [$HTTP_PROXY]	The HTTP proxy server URL to route traffic through
--help	See description in Global flags below

Command	Description
--version	Retrieves the version of the Cortex CLI currently in use

Global flags

The following table describes global CLI flags that are common specifically to the Application Security (AppSec) and Cloud Workload Protection (CWP) modules. These flags must be placed **before** the command.

Command	Description
--severity $CORTEX_CODE_SEVERITY	Filters scan results by severity level. Accepts one or more comma-separated values: unknown, low, medium, high, critical. Repeat the flag or use comma separation to specify multiple levels (for example, `--severity high,critical`). Constraint: Only effective when `--upload-mode` is set to `no-upload`. When upload mode is active, the flag is ignored and an informational message is displayed. **Important**: Severity filtering is currently only by the Application Security module
--upload-mode $CORTEX_UPLOAD_MODE	Controls whether scan results are uploaded to the Cortex Cloud platform. Accepts placement in both the global position (`cortexcli --upload-mode no-upload code scan`) and the command position (`cortexcli code scan --upload-mode no-upload`). The global position takes priority over the command position. Accepted values: • `upload`: Uploads results to the platform and triggers policy evaluation • `no-upload`: Executes scanners locally without uploading results. Enables `--severity` filtering • `no-code`: Uploads results without uploading source code

Command	Description
--soft-fail $CORTEX_SOFT_FAIL	See description in Common flags above
--no-fail-on-crash $CORTEX_NO_FAIL_ON_CRASH	Prevents the CLI from returning a non-zero exit code during internal errors (such as scanner crashes or network timeouts), ensuring CI/CD pipeline continuity even if a scan fails. When to use: Enable `--no-fail-on-crash` in production CI/CD pipelines where build availability takes priority over scan enforcement. For example, if the Cortex Cloud platform experiences a temporary outage, --no-fail-on-crash prevents the outage from blocking all builds across the organization. Combine with --log to ensure that internal errors are still captured for post-incident review. Signal-based exit codes (126, 127, 128+) indicating the CLI itself could not execute are never suppressed and require immediate investigation. Important: The environment variable changed from $CORTEX_CODE_NO_FAIL_ON_CRASH to $CORTEX_NO_FAIL_ON_CRASH as part of the framework-level migration. Update CI/CD pipeline configurations that reference the previous variable name.
--log	Displays the path to the log file after command execution. Use this to troubleshoot CI/CD failures or provide details for support cases. By default, logs are stored at `~/.cortexcli/cortexcli-log/`. Log rotation: Includes automatic log rotation (10 MB per file, 3 backups, 24-hour retention).

Command	Description
--help	Displays usage information, available subcommands, global flags, and flag descriptions for the Cortex CLI or any specific subcommand. Run `--help` at any level of the command hierarchy to discover available options: • cortexcli --help: Lists all available modules (AppSec, CWP, WAAS), global flags, and getting-started guidance. • Authentication: The --help flag works without API key authentication. No credentials, network connectivity, or platform access are required. This allows developers to explore the CLI interface before configuring authentication

1.6 | Cortex CLI for API Security

API Security testing is implemented in Cortex Cloud through the Cortex CLI.

This testing evaluates APIs for vulnerabilities and misconfigurations using fuzzing techniques to ensure secure data transmission, prevent unauthorized access, and to ensure that the API behaves as expected under unexpected or malformed input.

> **PREREQUISITE:**
>
> - Ensure you have the required user permissions. Refer to Cortex CLI for more information
> - Onboard and install the Cortex CLI. Refer to Connect Cortex CLI for more information
> - Ensure your application exposes APIs and provides a corresponding OpenAPI Specification file
> - Ensure that you have installed `Java v 11` and above

Authentication

The authentication file schema defines the authentication method (such as JWT, Basic) used to authorize connections to your scanned application. The following example provides configurations examples for common methods, including Basic authentication, API Keys and bearer tokens.

Example 163. Authentication File Schema Example

```
type: headers
creds:
    name: <header name>
    value: <header value>
-----------------------------------
For basic auth
```

```
type: basic
creds:
    username: {USERNAME}
    password: {PASSWORD}
-------------------------------------
For API Keys
type: headers
creds:
    name: x-api-key
    value: {API key}
-------------------------------------
For Bearer tokens
type: headers
creds:
    name: Authorization
    value: Bearer {BEARER_TOKEN}
```

Running API Security scans

To scan API Security, run:

```
       ./cortexcli  --log-level <ERROR LEVEL> --api-base-url <API URL> --api-key <API key from the
"Authenticate" step in the CLI connector screen> --auth-id 1 api scan  --api-spec-file <OPENAPI SPEC LOCATION>
  --scanned-app-url <BASE URL OF THE SCANNED APP> --java-location <JAVA BIN LOCATION>
```

Output

The API Security scan generates a detailed scan report that includes:

- Findings: These include vulnerabilities and risks identified in the scanned application's APIs, such as SQL Injection, sensitive data leaks, and other issues
- Errors: This section lists error responses returned by the scanned application
- Metadata: Information such as runtime details, scan status (success or failure), scan duration, hostname and scan parameters

The following schema defines the structure and format of API Security scan reports.

Read more...

```
{
  "reportID": "string",
  "results": [
    {
      "id": "string",
      "name": "string",
      "description": "string",
      "url": "string",
      "method": "string",
      "risk": "string",
      "alert": "string",
      "tags": {},
      "statusCode": "integer",
      "requestBody": "string",
      "curlCommand": "string"
    }
  ],
  "serverErrors": [
    {
      "id": "string",
      "name": "string",
```

```
      "description": "string",
      "url": "string",
      "method": "string",
      "risk": "string",
      "alert": "string",
      "tags": {},
      "statusCode": "integer",
      "requestBody": "string",
      "curlCommand": "string"
    }
  ],
  "scanStartTime": "string (ISO 8601 datetime)",
  "elapsedSeconds": "number",
  "hostname": "string",
  "scanStatus": "string",
  "parameters": {
    "scannedAppURL": "string",
    "apiSpecFile": "string",
    "apiSpecType": "string",
    "timeoutSeconds": "integer"
  }
}
```


The following is an example of API Security scan output.

Read more...


```
{
  "reportID": "0a739ae6-d18e-11ef-8a06-263731778ec0",
  "results": [
    {
      "id": "0",
      "name": "Server Leaks Version Information via \"Server\" HTTP Response Header Field",
      "description": "The web/application server is leaking version information via the \"Server\" HTTP response
header. Access to such information may facilitate attackers identifying other vulnerabilities your
web/application server is subject to.",
      "url": "http://localhost:5000/api/v1/extract",
      "method": "POST",
      "risk": "Low",
      "alert": "Server Leaks Version Information via \"Server\" HTTP Response Header Field",
      "tags": {
        "CWE-200": "https://cwe.mitre.org/data/definitions/200.html",
        "OWASP_2017_A06": "https://owasp.org/www-project-top-ten/2017/A6_2017-Security_Misconfiguration.html",
        "OWASP_2021_A05": "https://owasp.org/Top10/A05_2021-Security_Misconfiguration/",
        "WSTG-v42-INFO-02": "https://owasp.org/www-project-web-security-testing-guide/v42/4-
Web_Application_Security_Testing/01-Information_Gathering/02-Fingerprint_Web_Server"
      },
      "statusCode": 404,
      "requestBody": "--d3b92f4f-e2e3-4caa-8b00-4e43c8df0d87\r\nContent-Disposition: form-data;
name=\"file\"\r\nContent-Type: text/plain\r\n\r\n\"John Doe\"\r\n--d3b92f4f-e2e3-4caa-8b00-4e43c8df0d87--",
      "curlCommand": "curl -X POST \"http://localhost:5000/api/v1/extract\" -H host: localhost:5000 -H user-
agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:125.0) Gecko/20100101 Firefox/125.0 -H pragma: no-cache -H
cache-control: no-cache -H accept: application/json -H content-type: multipart/form-data; boundary=d3b92f4f-
e2e3-4caa-8b00-4e43c8df0d87 -H content-length: 165 -d '--d3b92f4f-e2e3-4caa-8b00-4e43c8df0d87\r\nContent-
Disposition: form-data; name=\"file\"\r\nContent-Type: text/plain\r\n\r\n\"John Doe\"\r\n--d3b92f4f-e2e3-4caa-
8b00-4e43c8df0d87--'"
    }
  ],
  "serverErrors": [],
  "scanStartTime": "2025-01-13T11:09:04.919359+02:00",
  "elapsedSeconds": 1.349090375,
  "hostname": "My Computer",
  "scanStatus": "Failed",
  "parameters": {
    "scannedAppURL": "http://localhost:5000",
    "apiSpecFile": "openapi.json",
    "apiSpecType": "openapi",
```

```
    "timeoutSeconds": 300
  }
}
```

1.6.1 | Cortex CLI API Security command line reference guide

This reference guide describes the dedicated API Security commands and flags, including the structure of base commands and subcommands. Refer to Cortex CLI common command line reference guide for Cortex CLI commands common to all supported modules.

Value	Command
--scanned-app-url (string)	Base URL of the app to scan (required)
--api-spec-file (string)	Path to the API specification file (required)
--api-spec-type (string)	Type of the API specification ('openapi) (default "openapi")
--auth-file (string)	Path to the authentication file (optional). For more information on authentication, refer to Cortex CLI for API Security
--concurrency (int)	Concurrency limit for scan requests (default 5)
--java-location (string)	Path to the Java (version >= 11) binary file (default: Java)
--no-publish (boolean)	Avoid publish results to Cortex
--output-file (string)	Output path for the report file (optional)

Value	Command
--timeout (int)	Scan timeout in seconds (default 300)
--zap-port (int)	Listening port to be used by ZAP (default 35391)

1.7 | Cortex CLI for Cloud Workload Protection

Integrate Cloud Workload Protection (CWP) scans for secrets, vulnerabilities and malware during your continuous integration (CI) process. By leveraging Software Bill of Materials (SBOM) analysis, you can identify and remediate vulnerabilities before images are pushed to the registry, shifting security left and reducing risk in your cloud environments.

> **PREREQUISITE:**
>
> - Ensure you have the required user permissions. Refer to Cortex CLI for more information
> - Onboard and install the Cortex CLI. Refer to Connect Cortex CLI for more information
> - Verify that `Java` version 11 and above is installed: Run `java –version` in your terminal. If not, refer to Java SE Development Kit 11.0.25 for information about installing Java

Run CWP security scans

The `cortexcli image scan` command allows you to perform CWP scans on container images. By default, `cortexcli` scans images directly from your local Docker daemon's repository. You can also specify an image archive file to scan instead.

Prerequisite: Before you begin, ensure you have `sudo` privileges to execute the image scan.

> **NOTE:**
>
> CWP does not support container image secret scanning for systems running on ARM architecture.

Scan from local Docker daemon

For direct scanning from your Docker daemon, the image must already exist in your local Docker repository. The CLI will not pull a new image if it does not exist locally.

To scan an image that exists in your local Docker daemon, simply provide its name:

```
./cortexcli --api-base-url <API URL> --api-key <API key from the "Authenticate" step in the CLI connector screen> --api-key-id <API key ID from the "Authenticate" step in the CLI connector screen> image scan <image name>
```

The image scan accepts the following arguments:

- `--api-base-url`: Required - true. The public facing API URL. Refer to Connect Cortex CLI for more information
- `--api-key`: Required - true. Your Cortex Cloud API key. Refer to Connect Cortex CLI for more information
- `--api-key-id`: Required - true. Your Cortex Cloud API key ID
- `image scan`: Required - true. Refers to CWP as the type of scan

> **NOTE:**
>
> For available CWP commands, refer to Cloud Workload Protection command line reference.

Example 164. EXAMPLE

```
./cortexcli --api-base-url https://api.cortex.example.com --api-key your-api-key --api-key-id 1 image scan
docker.io/library/nginx:latest
```

Example 165. EXAMPLE with custom Docker socket path

```
./cortexcli --api-base-url https://api.cortex.example.com --api-key your-api-key --api-key-id 1 image scan --
docker-host unix:///var/snap/docker/common/run/docker.sock my-custom-image:latest
```

By default, Cortex Cloud looks for the Docker socket at `unix:///var/run/docker.sock`.

`--docker-host <path>` specifies the path to the Docker socket. Use this flag if your Docker socket is located elsewhere, for example `unix:///var/snap/docker/common/run/docker.sock`.

Scan from an image archive file

> **PREREQUISITE:**
>
> Before you begin, ensure you have sudo privileges to execute the image scan.

To scan an image from a previously saved archive file (such as a .tar file), use the --archive flag:

```
./cortexcli --api-base-url <API URL> --api-key <API key from the "Authenticate" step in the CLI connector
screen> --api-key-id <API key ID from the "Authenticate" step in the CLI connector screen> image scan --archive
<archive file of container image>
```

> **NOTE:**
>
> - `--archive`: When used with image scan, sets the scan source to an archive file. When used with image sbom, indicates the SBOM should be exported from an archive file
> - The `--archive` flag can also be explicitly set as `--archive=true`
> - `--archive-format <value>`: The image archive format (such as `docker-archive` or `oci-archive`). Default: `docker-archive`.

Create an image archive

This example demonstrates how to create an image archive from your Docker or Podman environment, which can then be used for scanning or SBOM generation if you choose not to scan directly from the local daemon.

- With Docker: `docker save -o ubuntu.tar ubuntu`
- With Podman: `podman save --format oci-archive -o /tmp/alpine-oci.tar alpine:latest`

Export SBOM

You can generate a Software Bill of Materials (SBOM) for your container images using the Cortex CLI and and save the output to a specified file. This functionality enables you to store the SBOM for further analysis, auditing, and compliance.

By default, this will retrieve the SBOM for an image from your local Docker daemon.

To get an SBOM for an image from your local Docker daemon:

```
./cortexcli --api-base-url <API URL> --api-key <API key from the "Authenticate" step in the CLI connector screen> --api-key-id <API key ID from the "Authenticate" step in the CLI connector screen> image sbom <image name> [command options]
```

Command: `cortexcli image sbom`: Exports a Software Bill of Materials (SBOM) document for a container image archive.

Usage: `cortexcli image sbom [command options]`

Options:

- `--archive-format value`: Specifies the image archive format. Values: `docker-archive` (default), `oci-archive`
- `--output-format value`: Specifies the SBOM document output format. Values: `json` (default), `xml`
- `--output-file value`: Specifies the path to the file where the SBOM document will be saved
- `--fields value [--fields value]`: Specifies the fields to include in the SBOM document. Multiple fields can be specified including: author, binaries, license, name, purl, sourcePackage, type, version
- `--help`, `-h`: Displays help information for the command

Example 166. EXAMPLE

```
./cortexcli --api-base-url https://api.cortex.example.com --api-key your-api-key --api-key-id 1 image sbom docker.io/library/alpine:latest
```

To export an SBOM from an image archive file, use the `--archive` flag:

```
./cortexcli --api-base-url <API URL> --api-key <API key from the "Authenticate" step in the CLI connector screen> --api-key-id <API key ID from the "Authenticate" step in the CLI connector screen> image sbom --archive <archive file of container image>
```

NAME: `cortexcli image sbom` - Exports an SBOM document for an image from the local Docker daemon or an image archive.

USAGE: `cortexcli image sbom` [command options] [image name or archive file].

Troubleshooting

- Docker socket not reachable: If you encounter errors indicating the Docker socket cannot be reached, ensure the Docker daemon is running and verify the path to your Docker socket. If it's not in the default location (`unix:///var/run/docker.sock`), use the `--docker-host` flag to specify the correct path
- Image not found: If you attempt to scan an image directly from the Docker daemon and receive an error that the image does not exist, confirm that the image is indeed present in your local Docker repository by running `docker images`. The CLI will not pull images

1.7.1 | Cloud Workload Protection command line reference

This reference guide documents the Cloud Workload Protection commands and flags for the Cortex CLI, including the structure of base commands and subcommands. Refer to Cortex CLI common command line reference guide for Cortex CLI commands common to all supported modules.

Read more...

Command	Description
--image scan	Scans a container image archive
--ci-pipeline-id value	The CI pipeline identifier
--ci-build-id value	The CI build identifier
--timeout value	Timeout (in seconds) after which the scan will be terminated if it has not completed (default: 60)
--output-format value	Output format options: `human-readable`, `json` (default: human-readable)
--archive-format value	The image archive format options: `docker-archive`, `oci-archive` (default: docker-archive)
--name value	The name assigned to the image
--docker-host <path>	Specifies the path to the Docker socket

Command	Description
--archive	Specifies that the image scan should use an archive file

1.8 | Cortex CLI for Code Security

Cortex CLI for Code Security scans allow developers and security teams to integrate security checks directly into their application development workflows.

The Code Security CLI supports the following scan types:

- Secrets: Identifies exposed sensitive secrets within your codebase
- Infrastructure-as-Code (IaC): Analyzes infrastructure configuration files to detect potential security misconfigurations
- Software Composition Analysis (SCA): Performs vulnerability detection in third-party dependencies, assesses their license compliance and their package operational risk

In addition, the Code Security CLI serves as the integration mechanism for security scanning within supported CI tools such as Jenkins, GitHub Actions, and others. This is achieved by adding a code snippet containing the CLI command into the configuration files of your CI tool when integrating the CI tool with Cortex Cloud. It acts as a wrapper, enabling security scanning within your pipelines, and direct upload of results to the platform.

The CLI supports the following outputs:

- json
- spdx
- cli
- junitxml
- sarif
- cyclonedx
- cyclonedx_json

Code Security CLI scan behavior and output

- Scans generate assets (see Code Security assets, issues, and findings
- If one scanner (such as Secrets) fails, the other scanners will continue to run and produce results
- Scan failures trigger an error message indicating the scanner that failed
- The Code Security CLI provides these output modes for flexible management and viewing of scan results:
    - Upload to platform: `--upload-mode = true` (default). Uploads scan results directly to the platform for centralized analysis and management
    - Upload findings only. `--upload-mode = false` (default). Upload findings, but without including the actual source code content. This prevents raw source code from leaving your local environment or being stored on the platform
    - CLI output only: `upload = false` (default). View scan results directly in your command-line interface without being uploaded to the platform

  For more information about the output flags, refer to Cortex CLI Cortex Cloud Application Security command line reference.

Authentication

To authenticate the Code Security CLI, choose one of the following methods:

- Local developer workflows: Run manual, ad-hoc scans on your local machine to catch vulnerabilities and misconfigurations before committing code to your version control system

  The following flags are required to authenticate the Code Security CLI:

    - `--api-base-url`: [$CORTEX_API_BASE_URL]
    - `--api-key`: [$CORTEX_API_KEY]
    - `--auth-id`. [$CORTEX_AUTH_ID]

  For more information about these flags, refer to Cortex CLI common command line reference guide.

- Using a `cortex.env` file: Place your authentication details in a `cortex.env` file. You can download this file from the UI
- CI/CD pipeline automation: The Application Security CLI serves as the core integration mechanism for security scanning within your automated pipelines. By inserting simple code snippets into CI tools like Jenkins, GitHub Actions, CircleCI, or GitLab Runner, the CLI acts as a wrapper to enforce security guardrails dynamically and block risky deployments

Requirements

> **PREREQUISITE:**

- **For the Cortex CLI binary**:
  - Ensure you have `Node.js v22` installed on your host machine before running any scans with the Cortex CLI. This is crucial to prevent runtime errors, as the CLI depends on Node.js for executing JavaScript analysis

    > **NOTE:**
    >
    > - To check your version of `Node.js`, run `node -v`
    > - To download Node.js, refer to the official Node.js site

  - For Linux OS systems, ensure that GLIBC (GNU C library) version 2.35 or greater is installed

  > **NOTE:**
  >
  > This requirement does not apply when using the CLI as a container image.

- **Permissions**: Ensure you have the required user permissions. Refer to Cortex CLI for more information
- Onboard and install the Cortex CLI. Refer to Connect Cortex CLI for more information

Configure proxy for the Code Security CLI

When operating the Code Security CLI within environments requiring internet access via a proxy server, you can configure the tool to route its traffic through your proxy using standard environment variables. For proxies that perform TLS inspection, you must also specify a CA certificate

- Environment variables: Set `HTTP_PROXY` and `HTTPS_PROXY` (or `http_proxy` and `https_proxy`) to your proxy address
- CA Certificate: Use the `--ca-certificate` flag or the `$CORTEX_CA_CERTIFICATE` environment variable to provide your CA certificate for proxies that perform TLS inspection. The flag is now global and must appear before `code scan`. It is currently limited to the Application Security CLI. You can either:

1.8.1 | Cortex CLI usage for Cortex Cloud Application Security

To scan Cortex Cloud Application Security, run:

```
cortexcli --api-base-url <API URL> --api-key <API key from the "Authenticate" step in the CLI connector screen>
--api-key-id <API Key ID> code scan --directory {{DIRECTORY}} --branch main --repo-id organization/repo-name
-output json --output-file-path ./output.json
```

Command line reference

The command structure includes global flags which are used for authentication, and then specifies the module name and command specific to Cortex Cloud Application Security which are followed by dedicated flags unique to this module as well as flags common to all modules.

- Global flags: These flags are part of the initial `cortexcli` command and are necessary to authenticate and connect to Cortex Cloud
    - `--api-base-url`: (Required = true). The public facing API URL. Refer to Connect Cortex CLI for more information
    - `--api-key`: (Required = true). The Cortex Cloud API key generated when onboarding the CLI as a data source. Refer to Connect Cortex CLI for more information
    - `--api-key-id`: (Required = true). The Cortex Cloud API key ID generated when onboarding the CLI as a data source

  For a comprehensive list of Cortex Cloud Application Security global flags, refer to Cortex CLI Cortex Cloud Application Security command line reference
- Cortex Cloud Application Security specifics: Following the global flags, the command specifies the module and the commands required for initiating a scan using the Cortex Cloud Application Security module:
    - `code scan`: Required - true. This command instructs the CLI to perform an Cortex Cloud Application Security scan.
    - For the optional flags, refer to the dedicated Cortex Cloud Application Security command line reference

CLI Usage Examples

- Send output to a file: Direct the command's output to a specified file instead of displaying it in the console

```
./cortexcli --api-base-url <BASE_URL> --api-key <API_KEY> --api-key-id <API_KEY_ID> code scan --branch <branch name> --repo-id <repo name> --directory <path> --output json --output-file-path <path>
```

- Perform a scan without upload: Run a scan for local analysis or testing without uploading the results to Cortex Cloud. This command runs a code scan and saves all standard output (human-readable format) to `scan_results.txt`

```
./cortexcli --api-base-url <BASE_URL> --api-key <API_KEY> --api-key-id <API_KEY_ID> code scan --upload-mode no-upload --branch <branch name> --repo-id <repo name> --directory <path>
```

Sample outputs

The `cortexcli` provides different options for how scan results are presented.

- Standard output (stdout): When no specific output format flags (such as `--output json` or `--output sarif)` are provided, the Cortex CLI will produce standard output directly to your terminal or console
- JSON output: To obtain the output of a scan command as a JSON file, specify the flags `--output json --output-file-path ./output.json`. This command will save the detailed scan results in JSON format to output.json in the current directory.

Supported flags

The Cortex Cloud Application Security CLI supports both common Cortex CLI and dedicated Cortex Cloud Application Security flags.

- For dedicated Cortex Cloud Application Security flags, refer to Cortex CLI Cortex Cloud Application Security command line reference
- For common flags, refer to Cortex CLI common command line reference guide

1.8.2 | Cortex CLI Cortex Cloud Application Security command line reference

This reference guide documents the commands and flags unique to the Cortex Cloud Application Security CLI. For CLI commands common to all supported modules refer to Cortex CLI common command line reference guide.

> **IMPORTANT:**
>
> The Cortex CLI Cortex Cloud Application Security only supports single occurrences of each flag. If the same flag is passed multiple times, only the last provided value will be used. For example, in the following command, only TF CloudFormation will be the scanned framework.
>
> Example 167.
>
> ./cortexcli --api-base-url <YOUR_API_URL> --api-key <YOUR_API_KEY> --auth-id <YOUR_AUTH_ID> --framework terraform --framework "terraform cloudformation"

Command/Variable	Description
--source $CORTEX_CODE_SOURCE	The source of execution. Default source: CLI. Examples: Jenkins, GitHub Actions, CLI

Command/Variable	Description
--repo-id $CORTEX_CODE_REPO_ID	Required for upload mode. Identity string of the repository. Format `repo_owner/repo_name`. **NOTE:** The repo-id flag must not end with `.config`, `.log` or `.ini`. `–config` is acceptable. Example 168. • `––repo–id foo.config` will be blocked • `––repo–id foo–config` will pass To retrieve the repository ID, under Inventory, navigate to All Assets → Repositories (under Code) → select a repository → copy the Asset ID value from the Properties section of the side card.
--branch $CORTEX_CODE_BRANCH	Required for upload mode. Path to custom CA certificate (bundle) file for corporate proxy/TLS interception environments
--directory $CORTEX_CODE_DIRECTORY	Required. The directory path to scan. Cannot be used together with `––file`
--file $CORTEX_CODE_FILE	The file path to scan. Cannot be used together with `––directory`. When using this option, the Cortex CLI will filter runners based on the file type provided. For example, if you specify a `.tf` file, only the Terraform and secrets frameworks will be included. You can further limit this (for example; skip secrets) by using the `––skip–framework` argument

Command/Variable	Description
--var-file $CORTEX_CODE_VAR_FILE	Variable files to load in addition to the default files. This feature is currently supported for both source Terraform (.tfvars files) and Helm chart scans (for providing custom values or variable overrides). Refer to https://www.terraform.io/docs/language/values/variables.html#va definitions-tfvars-files) for more information
--framework $CORTEX_CODE_FRAMEWORK	Filter to scan specific frameworks. Example: `--framework arm`. Syntax: Use a single flag with comma-separated values for multiple frameworks. Both quoted (`"arm,ansible"`) and unquoted (`arm,ansible`) formats are supported. Example: `--framework arm,ansible`. Constraint: Do not use multiple --framework flags: `--framework terraform --framework sca_package`. Environment variables: `export CORTEX_CODE_FRAMEWORK=arm,ansible`. Supported frameworks: `ARM`, `ANSIBLE`, `BICEP`, `CLOUDFORMATION`, `DOCKER`, `DOCKERFILE`, `HELM`, `KUBERNETES`, `KUSTOMIZE`, `OPENAPI`, `SCA`, `SECRETS`, `SERVERLESS`, `TERRAFORM`, `TERRAFORMJSON`, `TERRAFORMPLAN`
--skip-framework $CORTEX_CODE_SKIP_FRAMEWORK	Skip specific frameworks. Example: `--skip-framework terraform`. Syntax: Use a single flag with comma-separated values for multiple frameworks. Both quoted (`"arm,ansible"`) and unquoted (`arm,ansible`) formats are supported. Example: `--skip-framework terraform, sca_package`. Constraint: Do not use multiple skip --framework flags: `--skip-framework terraform --skip-framework sca_package`. Environment variables: `export CORTEX_CODE_SKIP_FRAMEWORK="tf,sca"`

Command/Variable	Description
--ca-certificate $CORTEX_CODE_CA_CERTIFICATE	See common flags for more information
--no-cert-verify $CORTEX_CODE_NO_CERT_VERIFY / $NO_CERT_VERIFY	This flag disables TLS/SSL certificate verification (default: false). Skips TLS certificate verification when connecting to the API. Not recommended for production. Use only in test or development environments, as this reduces connection security
--summary-position $CORTEX_CODE_SUMMARY_POSITION	Sets the position for displaying the summary information relative to the findings. Values: `top`, `bottom`
--upload-mode $CORTEX_UPLOAD_MODE	Upload mode determines the method or mode used to upload data. See common flags for more information
--external-modules-download-path $CORTEX_CODE_EXTERNAL_MODULES_DOWNLOAD_PATH	Specifies the directory to download external modules to. Defaults to `.external_modules`
--output $CORTEX_CODE_OUTPUT	Output format for reporting. Supported formats: cli, json, spdx, junitxml, sarif, cyclonedx, cyclonedx_json
--output-file-path $CORTEX_CODE_OUTPUT_FILE_PATH	Specifies the output path for the scan result file
--deep-analysis $CORTEX_CODE_DEEP_ANALYSIS	Enables or disables deep analysis of the Terraform plan and related files
--repo-root-for-plan-enrichment $CORTEX_CODE_REPO_ROOT_FOR_PLAN_ENRICHMENT	Enriches Terraform plan findings by mapping them to their original `.tf` files

Command/Variable	Description
--skip-path $CORTEX_CODE_SKIP_PATH	Specifies a path (file or directory) that should be skipped during the scanning process. This option is useful for excluding specific files or directories that are not relevant to the scanning analysis, increasing the efficiency and accuracy of scan results
--create-repo-if-missing &CORTEX_CODE_CREATE_REPO_IF_MISSING	Determines whether the system should create a repository if it is missing. This option allows users to automate the creation of repositories as needed and ensure that all required repositories are available for scanning. For example, when running automated scans or integrating with version control systems, enabling `--create-repo-if-missing` can help maintain consistency and prevent disruptions due to missing repositories
--compact $CORTEX_CODE_COMPACT	Do not display code blocks in the output
--no-fail-on-crash $CORTEX_NO_FAIL_ON_CRASH	See common flags for a description
--var-file $CORTEX_CODE_VAR_FILE	Variable files to load in addition to the default files, Currently only supported for source Terraform (.tf file) and Helm chart scans
--validate-secret CCORTEX_APPSEC_VALIDATE_SECRETS	Validate detected secrets against their respective services to confirm they are active. By default, this feature is disabled. Set `CORTEX_APPSEC_VALIDATE_SECRETS = true` to enable it

Command/Variable	Description
--timeout	Sets the maximum time the Cortex CLI will wait for triggered local scan processes to complete. Default value: 15 minutes. Syntax: • To specify a duration: Use a numeric value followed by a unit (for example `--timeout 10m`) • Default unit: Numeric values entered without a unit are interpreted as seconds. For example, `30` is equal to 30 seconds. • Supported units: Milliseconds, seconds, minutes and hours
--start-commit	Starting commit hash for git history scanning (Git Hook flag)
--commit-list	Comma-separated list of commit hashes to scan (Git Hook flag)
--hook-event	Git hook event type - pre-commit (Git Hook flag)
--help	See common flags for a description

1.8.3 | Cortex CLI pre-commit hooks

Abstract

Integrate Application Security secrets scanner as pre-commit hooks into your workflows to scan for errors on your machine before local commits.

Integrate the Cortex Cloud Application Security secrets scanner as a pre-commit hook by installing the Cortex CLI. The scanner executes the hook locally before a commit. This setup ensures that secrets checks are enforced before any changes are committed.

When setting up pre-commit hooks, you can choose between local hooks and global hooks.

- Local: Installs the hook in the `.git/hooks` directory of the current repository, ensuring that Cortex Cloud secrets scans automatically run on your code before every commit
- Global: Installs the hook for all Git repositories on your machine, so Cortex Cloud secrets scans will automatically run on your code before every commit, regardless of the project

How to configure pre-commit hooks

> **PREREQUISITE:**
>
> These common prerequisites are required for all types of installation (both local and global) of the Cortex CLI pre-commit hook.
>
> - Ensure you have a license for Cortex Cloud Application Security
> - Install the Cortex Cloud CLI binary locally. Refer to Connect Cortex CLI for information about onboarding the CLI
> - Obtain Cortex Cloud API credentials (API Key ID and API Key) available from the CLI onboarding process (see above), and your API base URL. For more information on creating API keys, refer to https://docs-cortex.paloaltonetworks.com/r/Cortex-XSIAM-REST-API/Create-a-new-API-key
> - **Git**: You must have Git installed on your machine. For installation instructions, refer to the official Git website

1. Create a directory:

   ```
   mkdir -p ~/.cortexcli
   ```

2. Create a `.cortex.yaml` file in the `~/.cortexcli/` directory.
3. Open the `.cortex.yaml` file and add your Cortex Cloud API credentials and API base URL to the `yaml` file:

   - `CORTEX_API_BASE_URL`: <replace with the base API URL>
   - `CORTEX_API_KEY_ID`: <replace with API Key ID>
   - `CORTEX_API_KEY`: <replace with API Key>

   > **NOTE:**
   >
   > It is recommended you configure credentials for the Cortex CLI using a configuration file.

4. For local hooks: Install the Cortex CLI pre-commit hook package to set up a local hook for the current Git repository:

   > **PREREQUISITE:**
   >
   > For local installation: Install the **pre-commit** framework version 3.2.0 or greater. Refer to https://pre-commit.com/ for installation instructions.

   a.
   - For macOS, you can use Homebrew:

     ```
     brew install pre-commit
     ```

   - For other installations run:

     ```
     pip install pre-commit
     ```

   b. Navigate to the root of your repository → run the following command:

```
cortexcli code pre-commit install --mode local
```

5. For Global hooks: Install the Cortex CLI pre-commit hook package to set up hooks for all Git repositories on your machine.

   ```
   cortexcli code pre-commit install --mode global
   ```

   > **NOTE:**
   >
   > The **pre-commit** framework is not required for global mode.

References

To set up the Cortex CLI as a pre-commit hook on supported platforms, refer to the following official Git documentation for managing hooks:

- Git Hooks: A comprehensive guide on all available Git hooks, including `Pre-commit`: https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks
- Atlassian Git Tutorial: A tutorial that explains the purpose and usage of both local and server-side hooks, including `pre-commit`: https://www.atlassian.com/git/tutorials/git-hooks

1.8.4 | Pre-commit hook usage

You can run secrets checks on your code, customize its behavior using supported flags, and suppress detected secrets when required.

By default, Cortex CLI pre-commit hooks:

- Scan staged files only: The scan performs a quick and efficient check by only analyzing the changes you are about to commit, rather than the entire codebase
- Scan for secrets only: Pre-commit hooks support secrets scans only
- Do not upload results to the platform: All scan results are kept local to your machine, ensuring your data remains private

Command flag reference

Use the following flags with the `cortexcli code pre-commit` command to customize scanner behavior.

- `--ignore-existing-secrets`: Ignores secrets that already exist from a periodic scan (default: false) `[$CORTEX_CODE_IGNORE_EXISTING_SECRETS]`
- `--validate-secrets`: Checks if the secrets are valid (default: false) `[$CORTEX_CODE_VALIDATE_SECRETS]`
- `--skip-path`: Specifies a file or directory path to skip during the scan `[$CORTEX_CODE_SKIP_PATH]`
- `--compact`: Prevents the display of code blocks in the output (default: false) `[$CORTEX_CODE_COMPACT]`
- `--summary-position`: Determines whether the summary appears on top (before the check results) or on bottom (after the check results). (default: top) `[$CORTEX_CODE_SUMMARY_POSITION]`
- `--no-fail-on-crash`: Returns exit code 0 instead of 2 in case of a failure in the integration with the platform (default: false) `[$CORTEX_CODE_NO_FAIL_ON_CRASH]`
- `--help, -h`: Displays a help message with available options

Secrets suppression

You can suppress secrets directly within your code by adding a comment. This is useful for secrets that are intentionally included or a false positive and are not a security risk. Currently, suppression is not supported in `JSON` files.

The comment format is:

```
cortex:skip=<SECRET_ID>:<suppression justification>
```

Replace `<SECRET_ID>` with the specific ID provided in the scan output, and provide a brief explanation for why the secret is being suppressed. The comment syntax will depend on the file type.

Example 169. Example

Comments in a Dockerfile begin with (#). Note the comment in the After suppression code-block below.

- Before suppression:

  ```
  ENV SEC_1="ghp_3xyKmc3W7XanE82IKHJ3Z3AfHbV"
  ```

- After suppression:

  ```
  # cortex:skip=APPSEC_SECRET_43: Suppress this key for testing purposes
  ENV SEC_1="ghp_3xyKmc3W7XanE82IKHJ3Z3AfHbV"
  ```

1.8.5 | Cortex CLI pre-receive hooks

Abstract

Integrate the Application Security secrets scanner as a pre-receive hook into your workflows to scan for errors before code is accepted into your repository.

Integrate the Cortex Cloud Application Security secrets scanner as pre-receive hook into your workflows installing the Cortex CLI. The hook runs on the remote server before changes are pushed, allowing you to enforce checks before code is accepted into version control.

Supported version control systems: Pre-receive hooks are supported for GitHub Enterprise, GitLab self-managed, and Bitbucket Data Center. To setup pre-receive hook on these platforms refer to Setup on third-party platforms below.

Pre-receive hook workflow setup

1. Fulfill prerequisites.
2. Configure API credentials.
3. Install the pre-receive hook.
4. Setup the pre-receive hook on third party platforms.

Setup requirements

> **PREREQUISITE:**
>
> Before you begin, ensure you have:
>
> - **Administrator** access to the VCS server and console
> - A valid license for Cortex Cloud Application Security
> - The **Cortex Cloud CLI binary** or Docker image installed on the server (requires `GLIBC (GNU C library) version 2.35` or greater). Refer to Connect Cortex CLI for information about onboarding the CLI
> - **Cortex Cloud API credentials** (API Key ID and API Key) and your API base URL. For more information on creating API keys, refer to https://docs-cortex.paloaltonetworks.com/r/Cortex-XSIAM-REST-API/Create-a-new-API-key
> - **Git** installed on your machine. For installation instructions, refer to the official Git website

Configure credentials

It is recommended to configure credentials for the Cortex Cloud Application Security Cortex CLI using a configuration file, instead of embedding them directly in the hook script.

1. Create a directory:

   ```
   mkdir -p ~/.cortexcli/.cortex.yaml
   ```

   > **NOTE:**
   >
   > Make sure to create the directory under the home directory of the Linux user that runs the Git hooks. This user is typically not the root user.

2. Configure credentials: Open the `.cortex.yaml` file in the `~/.cortexcli/` directory and add the following configuration parameters:

   - `CORTEX_API_BASE_URL`: <API base URL>
   - `CORTEX_API_KEY_ID`: < API key ID >
   - `CORTEX_API_KEY`: < API key>

Setup on third-party platforms

To set up the Cortex CLI as a pre-receive hook on supported third-party platforms, refer to the official vendor documentation:

- GitHub Enterprise: About pre-receive hooks
- GitLab self-managed: Git server hooks
- Bitbucket Enterprise: Using repository hooks

Reference script

Use the script below as reference to extend or modify your existing pre-receive hooks in your VCS provider.

```
#!/usr/bin/env bash

# This script is used to run Cortex CLI in a pre-receive hook.

# Hide the update notice.
export CORTEX_HIDE_UPDATE_NOTICE=1

CORTEX_CLI="/usr/local/bin/cortexcli"
BASE_COMMAND="--api-base-url ${CORTEX_API_BASE_URL} --api-key-id ${CORTEX_API_KEY_ID} --api-key
${CORTEX_API_KEY} code pre-receive"
OPTIONAL_FLAGS=''

# Run cortex cli
${CORTEX_CLI} ${BASE_COMMAND:-''} ${OPTIONAL_FLAGS:-''}

exit_code=$?

exit $exit_code
```

1.8.6 | Pre-receive hook usage

The hook executes a script on every git push.

By default, Cortex CLI pre-receive hooks:

- Only scans code changes: It analyzes the code difference included in the pushed commits, not the entire repository
- Scans for secrets only: The analysis is focused on detecting sensitive information
- Does not upload results to Cortex Cloud: All scan results are kept local to your machine (on the server)

Understanding the script variables

- `CORTEX_CLI`: Defines the executable path, pointing to the absolute location of the `cortexcli` binary
- `BASE_COMMAND`: Assembles the core command string, including authentication flags (`--api-base-url`, `--api-key-id`, `--api-key`) and the primary command: `code pre-receive`. The use of `${...}` ensures authentication variables are injected as flag values
- `OPTIONAL_FLAGS`: An empty variable placeholder for adding optional runtime arguments

Command flag reference

Use the following flags with the pre-receive command to customize scanner behavior.

Example command structure:

```
$ cortexcli code pre-receive [options]
```

Option	Description
`--ignore-existing-secrets`	Ignores secrets that already exist in the periodic scan (default: false) `[$CORTEX_CODE_IGNORE_EXISTING_SECRETS]`
`--validate-secrets`	Checks if the secrets are valid (default: false) `[$CORTEX_CODE_VALIDATE_SECRETS]`
`--skip-path`	Specifies a file or directory path to skip during the scan `[$CORTEX_CODE_SKIP_PATH]`
`--compact`	Prevents the display of code blocks in the output (default: false) `[$CORTEX_CODE_COMPACT]`
`--summary-position`	Determines whether the summary appears on top (before the check results) or on bottom (after the check results). (default: top) `[$CORTEX_CODE_SUMMARY_POSITION]`
`--no-fail-on-crash`	Returns exit code `0` instead of `2` in case of a failure in the integration with the platform (default: false) `[$CORTEX_CODE_NO_FAIL_ON_CRASH]`
`--help, -h`	Displays a help message with available options

Breakglass: Bypassing the hook

The breakglass feature allows you to intentionally bypass the pre-receive hook security scan. This is useful in urgent situations where a push must go through immediately, but it should be used with caution as it overrides your security policies.

1. Configure your server to accept custom push options:

```bash
git config receive.advertisePushOptions true
```

2. Add the `-o breakglass` option to your `git push` command:

```bash
git push -o breakglass
```

Troubleshooting and recommendations

- Refer to the Cortex CLI for more information on the Cortex CLI.
- Modify the script as required based on the server running the VCS
- The Cortex CLI must be available on the server. This documentation does not describe the CLI installation process
- Update the Cortex CLI periodically
- Instead of adding the API URL and credentials directly in the script, consider creating a `~/.cortexcli/.cortex.yaml` configuration file (owned by the git user and group) with the following contents:

```
CORTEX_API_BASE_URL: <api base url>
CORTEX_API_KEY: <api key>
CORTEX_API_KEY_ID: <api key id>
```